COHERENCE DEPTH AND FIBONACCI SCALING IN INTENSIONAL TYPE THEORY

HALVOR LANDE

Draft affiliation to be supplied
e-mail address: hsl@awc.no

Abstract. We study a metatheoretic invariant of framework extension in type theory. The paper models the worst-case scaling of *foundational core extensions*: globally acting sealed additions such as universe hierarchies, global modalities, and heavily promoted generic interfaces whose public behavior must be specified against the active exported interface. We call the least historical depth at which the resulting obligations stabilize the coherence depth of the foundation. We formulate a payload-aware trace principle decomposing each exported interface into new core payload plus resolved coherence trace and, in the cubical case, a telescopic subsumption lemma showing that the depth-two arity bound is realized by a genuine two-layer chronological window. Together these turn coherence depth into an affine recurrence law for integration debt. In extensional systems with UIP the depth is one. In the fully coupled cubical univalent setting considered here the depth is two, and under uniform payload size the resulting integration law is shifted Fibonacci. We also give a Cubical Agda mechanization of both the depth theorem package itself and its affine recurrence consequence.

1. Introduction

1.1. **The metatheoretic question.** This paper concerns a simple but surprisingly rigid question.

> When a type-theoretic core calculus is extended by sealing a new structure against the existing interface, how does the cost of integration scale with the historical depth of the library?

The question is not about transparent user developments inside a fixed library. It is about *sealed* extensions: new type formers, modalities, or promoted interface packages whose public behavior becomes part of the global exported interface. More specifically, the paper studies the worst-case regime of *foundational core extensions*: globally acting additions whose exported payload must be specified across the currently active interface. Examples include extending a univalent universe hierarchy, where the decoding machinery must be updated against the library's active codes; adding a global cohesive modality, which must distribute across the exported type formers and transports already present; and heavily promoted generic interfaces whose action is intended to range over the whole library. In that

Key words and phrases: intensional type theory, cubical type theory, coherence, univalence.

setting, extension is constrained not only by local typing rules, but also by the obligation to interact coherently with the eliminators, transports, computation principles, and comparison data already exported by the library.

Orthogonal extensions are not the subject of this recurrence law. Adjoining a new datatype such as `Tree` to a library that already contains `List` and `Socket` need not introduce primitive `Tree`–`Socket` interaction data merely to preserve canonicity. Such sparse or local extensions are important in practice, but they do not exhibit the maximal coupling pattern that this paper isolates. Ordinary user-level library growth therefore forms a sparse dependency DAG in general; the recurrence theorem studies only the asymptotic foundational-core regime of monolithic, globally acting extensions.

The central observation is that these obligations are stratified by historical depth. Some are purely unary: the new layer must act on one previously exported generator. Others are genuinely binary: the new layer must preserve a previously exported compatibility witness between two generators. The main invariant of the paper measures the least depth at which this process stabilizes.

1.2. **The core thesis.** We call that invariant the *coherence depth* (or *coherence window*) of the foundation. The thesis of the paper is that, once coherence depth is isolated, the asymptotic law of integration becomes forced.

> In any fully coupled univalent core calculus with uniform payload size, framework extension debt exhibits Fibonacci scaling.

Here "fully coupled" refers only to the foundational-core regime just described: admissible extensions are globally acting payloads whose semantics or promoted action is intended to range over the whole active interface, not orthogonal additions that leave most earlier exports untouched. Within that regime, every admissible extension must provide primitive interaction data against the whole active exported interface rather than against a chosen fragment, and maximal interface density is the canonicity-preserving form of that library-wide action. In extensional systems with UIP, the coherence depth collapses to 1, and under uniform payload size the recurrence is first-order affine. In the cubical univalent setting considered here, the coherence depth is 2, and after a constant payload shift the recurrence becomes Fibonacci.

1.3. **Summary of contributions.** The paper develops one theorem package with three components.

(1) We formalize *core framework evolution*: states, sealed extensions, promoted interface packages, active interfaces, integration latency, and the distinction between orthogonal extensions and globally acting foundational core extensions. Within this framework we prove a maximal-interface-density theorem for the fully coupled foundational-core regime and a payload-aware *Integration Trace Principle*, decomposing the exported interface of a sealed layer into its new public payload and the resolved obligations discharged in sealing it.
(2) We define coherence depth and prove a sharp dichotomy. In extensional type theory with UIP, the depth is 1. In the fully coupled cubical univalent setting, the depth is 2. The depth-two statement is supported by three convergent arguments: a dimensional upper bound from weak ω-groupoid semantics and cubical Kan filling, a categorical lower bound from adjunction coherence, and a topological lower-bound family supplied

by clutching constructions for principal bundles over spheres. The passage from the dimensional bound to a genuine two-step time window is made explicit by a telescopic subsumption lemma using cubical Kan filling.

(3) We combine the trace principle with the coherence-depth theorem to obtain a universal affine recurrence for integration latency. Under constant payload size c, the depth-two case specializes to

$$\Delta_{n+1} = \Delta_n + \Delta_{n-1} + 2c,$$

which becomes homogeneous Fibonacci after the shift $U_n = \Delta_n + 2c$. The associated structural inflation factor still converges to the Golden Ratio.

The present version is a first full draft. The Cubical Agda development described in section 5 now formalizes the opacity discipline, the payload-aware affine recurrence, the UIP collapse to depth 1, the algorithmic subsumption of arity-3 obligations by cubical composition, and the lower bound against global depth-one collapse. The dimensional, categorical, and topological arguments are still written here in condensed proof form because they explain the theorem architecture and its examples, but the paper's main depth claims no longer rest on prose alone.

1.4. **Related work.** The higher-dimensional background comes from the interpretation of intensional identity types as weak ω-groupoids, developed by Lumsdaine [Lum10] and van den Berg and Garner [vdBG11]. The operational setting for the intensional side of our depth dichotomy is cubical type theory in the sense of Cohen, Coquand, Huber, and Mörtberg [CCHM18], together with higher inductive types and Kan composition [CHM18]. For mechanization we use Cubical Agda [VMA19].

The coherence-theoretic side is informed by Mac Lane's coherence theorem [Lan63]. We also use Stasheff's analysis of higher associativity [Sta63] and the well-foundedness approach to coherence of Kraus and von Raumer [KvR19]. Our contribution is to turn coherence from a qualitative structural property into a quantitative invariant of extension debt.

The topological family used for the lower bound is the classical clutching description of principal bundles over spheres; for standard bundle-theoretic background we refer to Husemoller [Hus94]. The univalent and cubical perspective sits in the general landscape of homotopy type theory and univalent foundations [Uni13].

2. Core framework evolution and canonicity

We fix a foundation together with a discipline of *sealed* extension. The library grows as a sequence

$$\mathcal{B}_0 \rightsquigarrow \mathcal{B}_1 \rightsquigarrow \mathcal{B}_2 \rightsquigarrow \cdots$$

of states, where each step adjoins one new sealed layer.

2.1. States, extensions, and promoted packages.

Definition 2.1 (State)**.** A *state* or *library* is a monotone context $\mathcal{B}$ closed under derivability. An evolution step

$$\mathcal{B}_n \rightsquigarrow \mathcal{B}_{n+1}$$

adjoins one sealed integration layer, denoted L_{n+1}.

Definition 2.2 (Candidate)**.** A *candidate* over a library state $\mathcal{B}$ is a pair

$$X = (X_{\mathrm{core}}, \mathcal{G}_{\mathrm{obl}}),$$

where X_{core} is the mathematical content proposed for addition and $\mathcal{G}_{\mathrm{obl}}$ is the family of atomic coherence obligations induced when X is sealed against the active interface of $\mathcal{B}$.

Definition 2.3 (Core payload size)**.** For a candidate X, let $P(X)$ be the finite set of public schemas contributed directly by X_{core} before any coherence witnesses against earlier layers are added. Its cardinality

$$\kappa(X) := |P(X)|$$

is the *core payload size* of X.

Definition 2.4 (Sealed extension)**.** A *sealed extension* is a candidate whose obligations have been discharged and whose resolved data are exported only through an opaque public interface. Future extensions may use those exports, but may not inspect the internal derivations that produced them.

Definition 2.5 (Promoted interface package)**.** A *promoted interface package* over $\mathcal{B}$ is a finite package of constructively irreducible generators and interaction data that is exported as one new sealed layer. Such a package need not modify the reduction rules of the kernel. What matters is that its public generators become part of the active exported interface for later sealing steps.

Definition 2.6 (Integration latency)**.** The *integration latency* of a candidate X over $\mathcal{B}$ is

$$\Delta(X \mid \mathcal{B}) := |\mathcal{G}_{\mathrm{obl}}|,$$

the number of atomic obligations that must be resolved before X can be exported as a sealed layer.

For a sealed layer L_k obtained from a candidate X_k, we write $S(L_k)$ for its set of exported interface schemas and

$$\kappa_k := \kappa(X_k)$$

for its core payload size. The indexing is chosen so that Δ_k denotes the sealing cost of L_k.

Proposition 2.7 (Transparent user-level code lies outside the recurrence)**.** *Let U be a family of transparent definitions, lemmas, or record packages over $\mathcal{B}$ whose symbols are definitionally reducible using the existing operational semantics and are not promoted to a new opaque exported layer. Then U contributes zero integration latency and does not alter the active interface seen by later sealing steps.*

Proof. Such a development creates no new irreducible interaction site against the exported interface. All reductions remain inherited from the pre-existing library, and no new opaque generators are added to the global interface telescope. Hence U is ordinary growth inside one state, not a state transition of the form $\mathcal{B}_n \rightsquigarrow \mathcal{B}_{n+1}$. □

Definition 2.8 (Foundational core extension)**.** A candidate X over $\mathcal{B}_n$ is a *foundational core extension* if its payload is intended to act globally on the currently active interface: each public generator in $P(X)$ is operationally or semantically incomplete until its behavior has been specified against every generator in the designated coupling footprint inside the active exported interface. Typical examples are extensions of a universe-decoding apparatus, global modalities whose action must distribute across existing type formers and transports, and heavily promoted generic interfaces whose exported action is library-wide.

Remark 2.9. Not every sealed extension is foundational in the sense of 2.8. A new datatype or API layer may be orthogonal to much of the existing library and therefore require only sparse interaction data. Such extensions may preserve canonicity without becoming globally coupled to earlier layers. The recurrence theorem of this paper is intentionally a worst-case statement about foundational core extensions, not a universality claim about all sealed library growth.

This scope restriction is essential for the density theorem below. Ordinary user-level additions typically enlarge a sparse DAG of modules or datatypes and do not force primitive interaction with unrelated earlier exports. The maximal-coupling statement proved here isolates the special foundational-core regime in which the new sealed layer is advertised as a *global* operation on the active interface, so that canonicity and global closure become exhaustiveness requirements rather than bookkeeping conventions.

2.2. **Maximal interface density as a canonicity requirement.** To speak about coupling to historical layers, it is convenient to parameterize the visible interface by depth.

Definition 2.10 (Historical interface)**.** For $k \geq 1$, the *historical interface of depth k* at stage n is

$$I_n^{(k)} := \biguplus_{j=0}^{k-1} S(L_{n-j}),$$

the tagged coproduct of the schemas exported by the most recent k sealed layers.

Definition 2.11 (Maximal interface density)**.** A foundational core candidate sealed against $I_n^{(k)}$ has *maximal interface density* if every generator exported by $I_n^{(k)}$ lies in its coupling footprint and receives exactly one primitive interaction datum from the new extension. For native type formers such a datum is a defining reduction clause or decoding equation; for global modalities it is a distribution or transport law; for promoted interface packages it is a generatorwise comparison, transport, or action witness.

Definition 2.12 (Fully coupled foundation)**.** A foundation is *fully coupled* if the admissible sealed extensions under study are foundational core extensions and each such extension is required to satisfy maximal interface density with respect to the active historical interface. Equivalently, the paper's admissible extensions in a fully coupled foundation are globally acting payloads whose coupling footprint is the whole active interface, not orthogonal additions with sparse dependency patterns.

The next theorem turns maximal interface density into a metatheoretic necessity for this globally coupled regime rather than a stylistic choice.

Theorem 2.13 (Canonicity forces maximal interface density for foundational core extensions)**.** *Let $\mathcal{B}_n$ be a library state in a fully coupled foundation, and let X be an admissible foundational core extension sealed against the historical interface $I_n^{(k)}$. If the extended theory is to preserve the global closure and canonicity discipline of the sealed library, then every generator of $I_n^{(k)}$ must receive exactly one primitive interaction datum from X. Consequently,*

$$\Delta(X \mid \mathcal{B}_n) = |I_n^{(k)}|.$$

Proof. By 2.12, the admissible candidates considered here are not arbitrary sealed additions but foundational core extensions whose payload is intended to act on the whole active interface. The theorem is therefore about that global regime only; orthogonal extensions of the kind noted in 2.9 are outside its hypotheses.

First consider a native global computational extension. In a Tarski-style universe presentation, suppose the active interface exports a code universe U and a decoding map $\mathsf{El} : U \to \mathsf{Type}$. If the new sealed layer extends the universe with a fresh code but the extended decoding machinery is not given the corresponding new primitive computation clause, then closed terms mentioning that code no longer reduce through El, so canonicity fails on the extended language. More generally, if a new global operator such as a modality, transport schema, or promoted eliminator package is advertised as acting on the whole active interface but omits some generator of $I_n^{(k)}$, then the extended reduction or comparison relation is partial on well-typed terms built from that generator. In compiler terms, the global action has failed the relevant exhaustiveness condition.

For globally promoted interface packages the same point appears at the level of exported semantics. If the new opaque package is meant to provide a library-wide action but omits the comparison witness for some previously exported generator, then later sealing steps cannot treat it as a closed global operation ranging over the whole library. The public action is only partially defined on the active interface, so the sealed calculus is not globally closed.

This proves the "at least one" direction for foundational core extensions. For the converse, the issue is not metaphysical uniqueness of higher structure but the single-clause discipline required for computation. For native global extensions, two distinct primitive defining clauses for the same head and the same prior generator would overlap and destroy confluence. For globally promoted packages, once the public source, target, and prior generator are fixed, a second primitive datum would advertise competing basic behavior for the same global action. Hence exactly one primitive clause is forced at each head/generator site, and the number of obligations is precisely the cardinality of the historical interface. □

2.3. **The integration trace principle.** In the fully coupled foundational-core regime, the preceding theorem identifies sealing cost with the size of the active interface. The next theorem explains how those resolved obligations reappear in the next layer's public export alongside the layer's own new mathematical payload.

Theorem 2.14 (Integration Trace Principle)**.** *Each sealed integration layer exports a disjoint union of two kinds of public schema: the new payload contributed by its core content and the trace schemas corresponding to its resolved obligations. Equivalently, if L_k is the layer obtained by sealing a candidate of core payload size κ_k and coherence cost Δ_k, then there is a canonical decomposition*

$$S(L_k) \cong S_{\mathrm{core}}(L_k) \uplus S_{\mathrm{tr}}(L_k)$$

with

$$|S_{\mathrm{core}}(L_k)| = \kappa_k, \qquad |S_{\mathrm{tr}}(L_k)| = \Delta_k.$$

Hence

$$|S(L_k)| = \kappa_k + \Delta_k.$$

Proof. There are two logically distinct sources of exported schemas.

First, the candidate's core content $X_{k,\mathrm{core}}$ contributes genuinely new public generators: a new type former, modality, promoted interface operator, or other primitive schema that did not previously exist in the library. By definition of core payload size, the number of these newly exported payload schemas is κ_k. These form $S_{\mathrm{core}}(L_k)$.

Second, by 2.13, sealing against the active interface requires exactly one primitive interaction datum for each active exported generator. Once such a datum is supplied and the layer is sealed, the datum no longer survives as an internal derivation visible to future extensions. It survives only through the public schema recording how the new payload acts on that older generator. These schemas form $S_{\mathrm{tr}}(L_k)$, and there is one of them for each resolved obligation. Hence $|S_{\mathrm{tr}}(L_k)| = \Delta_k$.

Because the layer is sealed, every public schema of L_k is of one of these two kinds: either it is a newly introduced payload symbol, or it is the exported trace of a resolved obligation attaching that payload to prior interface data. The two kinds are disjoint, since payload symbols are primitive generators introduced by the new layer itself, whereas trace schemas are indexed by the prior interface against which the new layer was sealed. Therefore the public export decomposes canonically as

$$S(L_k) \cong S_{\mathrm{core}}(L_k) \uplus S_{\mathrm{tr}}(L_k),$$

and the cardinality formula follows immediately. □

Remark 2.15. The trace principle counts kernel-level extensions and promoted interface packages uniformly. In both cases the public export has a payload part and a trace part. The only difference is the kind of primitive datum involved: for native extensions the payload contributes new reduction heads and the trace contributes defining clauses, whereas for promoted packages the payload contributes new exported operators and the trace contributes comparison or transport witnesses.

3. The coherence window concept and the depth dichotomy

3.1. Induced obligations and historical arity.

Definition 3.1 (Induced obligations)**.** Let $\mathcal{O}^{(k)}(X)$ be the set of normalized atomic obligations induced when candidate X is sealed against a historical interface of depth k. Since the interface grows cumulatively, there are inclusions

$$\mathcal{O}^{(1)}(X) \subseteq \mathcal{O}^{(2)}(X) \subseteq \mathcal{O}^{(3)}(X) \subseteq \cdots .$$

Definition 3.2 (Historical support and arity)**.** For a normalized obligation o, let $\mathrm{Supp}(o)$ be the set of sealed layers whose exported schemas occur irreducibly in the typing derivation of o after each prior layer has been collapsed to its opaque interface. Its *historical arity* is

$$a(o) := |\mathrm{Supp}(o)|.$$

Remark 3.3. Historical arity counts independent prior layers, not textual age. A direct lookup to an old exported theorem is still unary, because it lands in one opaque interface. Multi-layer debt arises only when the new layer must cohere simultaneously with several independent prior exports.

Lemma 3.4 (Arity-to-dimension correspondence)**.** *An irreducible obligation of historical arity k requires a k-dimensional coherence cell. Arity 1 produces substitution or transport data, arity 2 produces a naturality square or homotopy, and arity $k \geq 3$ would require a genuine higher filler.*

Proof. With one historical input, the new layer must supply a comparison morphism between its behavior and one prior export, so the obligation is one-dimensional. With two independent historical inputs, the witness compares two unary composites and therefore has the form of a square or homotopy. More generally, relating k independently supplied historical interfaces requires a k-dimensional filler between the corresponding composites. □

3.2. **Definition of coherence depth.**

Definition 3.5 (Coherence depth)**.** A foundation has *coherence depth* d if d is the least integer such that for every candidate X and all $k \geq d$ there is a canonical equivalence

$$\mathcal{O}^{(k)}(X) \simeq \mathcal{O}^{(d)}(X).$$

We also call d the *coherence window* of the foundation.

The point of 3.5 is that beyond depth d, deeper historical interaction contributes no new independent obligations. Once that stabilization has been upgraded to a genuine chronological window, the size of the primitive active interface controls the recurrence law for integration debt.

Remark 3.6. 3.5 is a stabilization statement about *obligation sets*. It should not be read as saying that a bound on coherence dimension automatically identifies the chronological layers that must still be inspected primitively. In particular, an arity-two obligation could in principle couple a new layer to a remote historical layer. To extract a genuine time window from a dimensional bound one needs an additional subsumption argument showing that such remote binary comparisons are already generated by recently exported traces. In the cubical univalent case, this passage from arity to chronology is supplied by 3.9.

3.3. **Theorem A: extensional systems collapse to $d = 1$.**

Theorem 3.7. *In Martin-Löf type theory with UIP, the coherence depth is* 1.

Proof. Under UIP, any two proofs of identity between fixed endpoints are equal. Hence every identity type is a set, and every higher identity type is a proposition; once inhabited, it is contractible. Consider any normalized coherence obligation of dimension at least 2. By 3.4, it corresponds to a higher identity witness. But in the presence of UIP such a witness is uniquely determined by its boundary and therefore carries no independent data. Thus every obligation of arity at least 2 collapses to data already fixed by its unary boundary. Equivalently,

$$\mathcal{O}^{(k)}(X) \simeq \mathcal{O}^{(1)}(X) \qquad (k \geq 1).$$

Since depth 0 would mean the absence of any interaction with the current interface, which is impossible for a nontrivial sealing step, the least stabilizing depth is 1. □

3.4. **Theorem B: fully coupled univalent systems have $d = 2$.** From now on we work in a fully coupled cubical setting in the style of Cohen, Coquand, Huber, and Mörtberg [CCHM18, CHM18], with univalence and Kan composition available in the ambient operational semantics. The next three ingredients are deliberately complementary: the dimensional argument gives the upper bound, while the categorical and topological families give independent lower bounds. Together they pin down exact depth 2, and the telescopic lemma makes explicit how the dimensional bound is realized as a genuine two-step time window.

One semantic clarification is important before the cubical argument begins. In univalent foundations, types behave as weak ω-groupoids, so the geometric space of fillers for a fixed higher boundary is not generally contractible; open 3-boxes can admit many fillers. The integration-latency invariant does not count that homotopy-theoretic space. It counts the burden of supplying new primitive, human-authored, domain-specific payload at sealing time. In cubical type theory, the operational semantics provide the primitive hcomp, which computes a specific syntactic filler for any well-typed open box. The claim below is therefore algorithmic rather than topological: arity-3 obligations introduce no new primitive payload because the calculus uniformly synthesizes one filler from already exported unary and binary boundary data.

A concrete cubical sketch. The abstract distinction between payload and trace can be made visible in a tiny Agda-shaped example without importing the full PEN framework. Suppose an older sealed layer exports a small Tarski universe with function codes and their decoding equation. A new cohesive layer contributes a genuinely new modal payload `Flat` together with its unit `eta`. Everything else needed to make that modality act on the older universe is integration trace: a lifted action on old codes, a comparison between decoding before or after lifting, and the distribution law over old function codes. In the snippet below, the witness `remote-face` is the point of the example. It is not additional payload. It is exactly the missing comparison face of the open 3-box determined by the old decoding law together with the new unary and binary trace data, and cubical composition computes it uniformly.

```
-- Schematic Cubical Agda: old universe + new payload + induced trace

record OldU : Set1 where
  field
    U      : Set
    El     : U -> Set
    Arr    : U -> U -> U
    El-Arr : (A B : U) ->
      Path Set (El (Arr A B)) (El A -> El B)

record Payload : Set1 where
  field
    Flat : Set -> Set
    eta  : {A : Set} -> A -> Flat A

record Trace (L : OldU) (P : Payload) : Set1 where
  open OldU L
  open Payload P
```

```
  field
    FlatU         : U -> U
    flat-El       : (A : U) ->
      Path Set (El (FlatU A)) (Flat (El A))
    flat-code-Arr : (A B : U) ->
      Path U (FlatU (Arr A B)) (Arr (FlatU A) (FlatU B))
    flat-Arr      : (A B : U) ->
      Path Set (Flat (El A -> El B))
               (Flat (El A) -> Flat (El B))

module _ (L : OldU) (P : Payload) (T : Trace L P) where
  open OldU L
  open Payload P
  open Trace T

  -- left route:
  --   El (FlatU (Arr A B))
  --     -> Flat (El (Arr A B))
  --     -> Flat (El A -> El B)
  --     -> Flat (El A) -> Flat (El B)

  -- right route:
  --   El (FlatU (Arr A B))
  --     -> El (Arr (FlatU A) (FlatU B))
  --     -> El (FlatU A) -> El (FlatU B)
  --     -> Flat (El A) -> Flat (El B)

  postulate
    left-route  :
      (A B : U) ->
      Path Set (El (FlatU (Arr A B))) (Flat (El A) -> Flat (El B))
    right-route :
      (A B : U) ->
      Path Set (El (FlatU (Arr A B))) (Flat (El A) -> Flat (El B))

  -- The open 3-box says these two boundary composites agree.
  -- This face is derived trace, not new payload.
  remote-face :
    (A B : U) ->
    Path (Path Set (El (FlatU (Arr A B)))
                   (Flat (El A) -> Flat (El B)))
         (left-route A B) (right-route A B)
  remote-face A B = hcomp ...
```

Read top-to-bottom, `Payload` is the human-authored new content, while `Trace` is the compatibility debt forced by sealing against the old universe. The two routes are the two composites from `El (FlatU (Arr A B))` to `Flat (El A) -> Flat (El B)`. The arity-3 obligation is precisely the comparison between those composites. In a cubical presentation that

comparison is not another primitive clause that must be postulated separately for each old constructor; it is the filler of the open box generated by the already exported boundary.

3.4.1. *Dimensional upper bound ($d \leq 2$).*

Theorem 3.8 (Dimensional upper bound)**.** *In the fully coupled cubical univalent setting, every irreducible coherence obligation has historical arity at most* 2.

Proof sketch. The argument has three stages.

First, direct references to older exports do not by themselves create higher-arity debt. By 3.3, a lookup into an old opaque layer is still unary.

Second, genuinely new sealing pressure is binary. An independent new obligation appears when the candidate must cohere not only with a prior operator but also with an already exported compatibility witness involving that operator. Such data form naturality squares or homotopies, hence live on the 2-skeleton.

Third, arity at least 3 contributes no new primitive payload. Identity types in intensional type theory carry weak ω-groupoid structure [Lum10, vdBG11], so one should not expect geometric uniqueness of higher fillers in general. However, the cubical operational presentation equips the calculus with Kan operations hcomp, comp, and transp that compute a specific filler term for any well-typed open box from its lower-dimensional boundary data [CCHM18, CHM18]. Therefore any apparent arity-k obligation with $k \geq 3$ is algorithmically synthesized from its unary and binary boundary trace and contributes no new primitive clause to sealing. No irreducible independent obligation survives beyond arity 2. □

Lemma 3.9 (Telescopic subsumption)**.** *In the fully coupled cubical univalent setting, primitive sealing against the two most recent layers algorithmically synthesizes all older binary comparisons. More precisely, let X be a candidate added at stage $n+1$. Once X has supplied its primitive unary and binary data against L_n and L_{n-1}, every would-be primitive comparison between X and a layer L_{n-r} with $r \geq 2$ is uniformly derived by iterated cubical Kan filling from the traces already exported by $L_n, \ldots, L_{n-r+1}$.*

Proof sketch. Start with the first nonadjacent layer L_{n-2}. Because L_n was itself sealed against L_{n-1} and L_{n-2}, its exported trace includes a unary comparison

$$f_n : L_n \to L_{n-1},$$

the previous layer exports

$$g_{n-1} : L_{n-1} \to L_{n-2},$$

and the binary part of the trace of L_n contains a 2-cell

$$\alpha_n : g_{n-1} \circ f_n \Rightarrow h_n$$

recording the induced comparison $h_n : L_n \to L_{n-2}$. When X is sealed against the last two layers, it supplies unary comparisons

$$x_n : X \to L_n, \qquad x_{n-1} : X \to L_{n-1},$$

and a binary coherence square

$$\beta_n : f_n \circ x_n \Rightarrow x_{n-1}.$$

The cells x_n, x_{n-1}, f_n, g_{n-1}, α_n, and β_n, together with the evident degenerate composition faces, determine an open 3-box. Its missing face is precisely a comparison

$$\gamma_{n-2} : h_n \circ x_n \Rightarrow g_{n-1} \circ x_{n-1}$$

between the two composites from X to L_{n-2}. Cubical Kan filling via hcomp computes a particular syntactic filler for that box, so γ_{n-2} is derived rather than primitive.

The same argument now iterates. Suppose inductively that a comparison from X to L_{n-r+1} has already been syntactically derived. Since L_{n-r+1} was sealed against L_{n-r} and L_{n-r-1}, its exported trace again contains the adjacent unary bridge to L_{n-r} together with the corresponding binary square witnessing the comparison of composites. Combining that previously exported square with the already derived comparison for X forms another open 3-box, and hcomp fills it again. The resulting derived face supplies the comparison from X to L_{n-r}. At each additional historical step, the older layer contributes boundary data only; it contributes no new primitive cell. Thus layers at distance at least two are telescopically subsumed by the adjacent traces already exported by the recent history. □

Corollary 3.10 (Two-layer chronological window)**.** *In the fully coupled cubical univalent setting, every sealing step at stage $n+1$ has a genuine chronological window of size 2: all primitive obligations are generated by the exported traces of L_n and L_{n-1}, while comparisons with layers L_{n-r} for $r \geq 2$ are algorithmically synthesized by iterated Kan filling. Equivalently, the pair (L_n, L_{n-1}) forms a chronological Markov blanket for subsequent sealing.*

Proof. 3.8 shows that no irreducible obligation has historical arity greater than 2. 3.9 upgrades that dimensional statement to a temporal one: any binary comparison with a remote layer is induced from the adjacent trace data already exported by the last two layers. Hence primitive sealing data are exhausted by $S(L_n) \uplus S(L_{n-1})$, and all deeper history contributes only derived fillers. □

3.4.2. *Categorical lower bound: the adjunction barrier ($d \geq 2$).*

Theorem 3.11 (Adjunction barrier)**.** *A depth-one system cannot verify the genuine coherence of a nontrivial adjunction. Consequently, any fully coupled univalent setting supporting adjoint structure has coherence depth at least 2.*

Proof sketch. An adjunction $L \dashv R$ is specified by functors L and R, together with a unit and counit

$$\eta : 1 \to RL, \qquad \varepsilon : LR \to 1,$$

satisfying the triangle identities

$$\varepsilon L \circ L\eta \simeq \mathrm{id}_L, \qquad R\varepsilon \circ \eta R \simeq \mathrm{id}_R.$$

The triangle identities are genuine 2-cells. A depth-one system can mention functors and natural transformations, but it cannot retain independent binary coherence data unless all higher cells collapse, as in the UIP situation of 3.7. In a fully coupled univalent setting the triangle identities are not trivial: they are part of the structure that later sealed layers may need to use. Hence depth one is insufficient, and $d \geq 2$. □

3.4.3. *Topological exact family: clutching constructions.*

Theorem 3.12 (Clutching family)**.** *For principal G-bundles over spheres presented by clutching functions, the required sealing coherence is exactly depth 2: a genuine binary comparison is necessary, but no independent depth-three cocycle survives.*

Proof sketch. A principal G-bundle over S^m is classified by a clutching map

$$\gamma : S^{m-1} \to G,$$

equivalently by a class in $[S^m, BG] \cong [S^{m-1}, G]$ [Hus94]. To seal the total space as an exported layer, one must compare the path geometry of the base sphere with the automorphism data of the fiber encoded by γ. This comparison is irreducibly binary: it is a 2-cell witnessing compatibility between base transport and fiber twisting. Hence depth one does not suffice.

On the other hand, the standard two-chart cover of the sphere by hemispheres has intersection the equator and no triple intersections. Consequently the clutching presentation contains the equatorial comparison but no independent depth-three Čech cocycle condition. Any apparent higher condition is induced from the equatorial binary comparison. Therefore the family furnishes an exact lower-bound witness at depth 2 with no new independent depth-three data. □

Corollary 3.13. *In the fully coupled cubical univalent setting considered above, the coherence depth is exactly 2.*

Proof. 3.8 provides the upper bound by showing that no independent obligation survives beyond the 2-skeleton. 3.9 shows that this depth-two upper bound is realized by a genuine two-layer chronological window rather than merely by a bound on cell dimension. 3.11 and 3.12 provide lower bounds by exhibiting genuine depth-two phenomena that do not collapse in the univalent setting. Therefore the least stabilizing depth is 2. □

4. The complexity scaling theorem

Let L_n denote the nth sealed layer and write

$$\Delta_n := \Delta(L_n \mid \mathcal{B}_{n-1})$$

for its sealing cost, and let κ_n be its core payload size. The recurrence theorem counts *primitive* sealing data, so it uses not only the stabilization statement of 3.5 but also a theorem identifying a genuine chronological window. In the depth-two cubical univalent case, this is exactly 3.10; in the UIP case, the proof of 3.7 shows that all non-unary comparisons collapse to their boundary, so the window is one layer. More generally, whenever a foundation has coherence depth d realized by a chronological window of size d, the primitive active interface at stage $n+1$ is the depth-d historical interface built from the *full* exports of the preceding layers:

$$I_n^{(d)} = \biguplus_{j=0}^{d-1} S(L_{n-j}).$$

For the initial bootstrap stages before the full window has been populated, we adopt the natural convention that nonexistent earlier layers contribute the empty interface.

4.1. **The universal affine recurrence.**

Theorem 4.1 (Universal affine recurrence)**.** *For every foundation whose coherence depth d is realized by a chronological window of size d, the integration latency satisfies*

$$\Delta_{n+1} = \sum_{j=0}^{d-1} \big(\Delta_{n-j} + \kappa_{n-j}\big) \qquad (n \geq d).$$

Proof. By definition, the next sealing cost is the size of the active interface:

$$\Delta_{n+1} = |I_n^{(d)}|.$$

Using the tagged coproduct decomposition of 2.10,

$$|I_n^{(d)}| = \sum_{j=0}^{d-1} |S(L_{n-j})|.$$

Now apply the Integration Trace Principle, 2.14, which gives

$$|S(L_{n-j})| = \Delta_{n-j} + \kappa_{n-j}$$

for each j. Substituting yields the claimed affine recurrence. □

For later comparison, write

$$\tau_n := \sum_{i=1}^{n} \Delta_i$$

for the cumulative integration latency through stage n.

4.2. **Affine one-step growth at $d = 1$.**

Corollary 4.2. *If the coherence depth is 1, this depth is realized by a one-layer chronological window, and every layer contributes the same core payload size c, then*

$$\Delta_{n+1} = \Delta_n + c.$$

If moreover the initial state exports no prior sealed layers, then

$$\Delta_1 = 0, \qquad \Delta_n = (n-1)c,$$

and therefore

$$\tau_n = c\,\frac{n(n-1)}{2}.$$

In particular, integration latency grows linearly and cumulative integration cost grows quadratically.

Proof. Substitute $d = 1$ into 4.1. Then only the term $j = 0$ remains, so

$$\Delta_{n+1} = \Delta_n + \kappa_n.$$

Under the uniform payload assumption $\kappa_n = c$, this becomes $\Delta_{n+1} = \Delta_n + c$. If the initial state exports no prior sealed layers, then the first sealing step sees the empty interface and therefore has cost $\Delta_1 = 0$. Induction gives $\Delta_n = (n-1)c$, and summing this arithmetic progression yields the stated formula for τ_n. □

4.3. **Affine Fibonacci scaling at $d = 2$.** By 3.10, the fully coupled cubical univalent depth-two theorem is chronologically realized by the two most recent sealed traces.

Assume in this subsection that every sealed layer contributes the same core payload size c, and write $(F_n)_{n\geq 1}$ for the Fibonacci sequence with $F_1 = F_2 = 1$.

Corollary 4.3 (Affine Fibonacci recurrence)**.** *In the fully coupled cubical univalent setting, if the coherence depth is 2 and every sealed layer contributes core payload size c, then*

$$\Delta_{n+1} = \Delta_n + \Delta_{n-1} + 2c.$$

If we define

$$U_n := \Delta_n + 2c,$$

then

$$U_{n+1} = U_n + U_{n-1}.$$

If moreover the initial state exports no prior sealed layers, then

$$\Delta_1 = 0, \qquad \Delta_2 = c,$$

so

$$U_n = cF_{n+2}, \qquad \Delta_n = c(F_{n+2} - 2),$$

and the cumulative latency satisfies

$$\tau_n = c(F_{n+4} - 2n - 3).$$

Proof. The recurrence is the case $d = 2$ of 4.1 together with the uniform payload assumption $\kappa_n = c$, giving

$$\Delta_{n+1} = \Delta_n + \Delta_{n-1} + 2c.$$

Adding $2c$ to both sides yields

$$U_{n+1} = U_n + U_{n-1}.$$

If the initial state exports no prior sealed layers, then the first layer has no earlier interface against which to cohere, so $\Delta_1 = 0$. The second layer sees only the first layer's export, whose size is $\kappa_1 + \Delta_1 = c$, hence $\Delta_2 = c$. Therefore

$$U_1 = 2c = cF_3, \qquad U_2 = 3c = cF_4.$$

Since both (U_n) and (cF_{n+2}) satisfy the same Fibonacci recurrence and agree at $n = 1, 2$, induction gives $U_n = cF_{n+2}$. Subtracting $2c$ yields

$$\Delta_n = c(F_{n+2} - 2).$$

Summing from $i = 1$ to n gives

$$\tau_n = c\sum_{i=1}^{n}(F_{i+2} - 2) = c(F_{n+4} - 2n - 3),$$

using the standard Fibonacci summation identity. □

Corollary 4.4 (Asymptotic inflation factor)**.** *Under the bootstrap and uniform-payload hypotheses of 4.3, the structural inflation factor*

$$\Phi_n := \frac{\Delta_n}{\Delta_{n-1}}$$

converges to the Golden Ratio

$$\varphi = \frac{1 + \sqrt{5}}{2}.$$

Proof. Under the bootstrap assumptions of 4.3,

$$\Phi_n = \frac{\Delta_n}{\Delta_{n-1}} = \frac{F_{n+2} - 2}{F_{n+1} - 2}.$$

The additive constants are asymptotically negligible relative to the Fibonacci growth, so the standard ratio convergence gives $\Phi_n \to \varphi$. □

5. Cubical Agda mechanization

5.1. **Mechanizing the theorem package.** For the purposes of the present paper, the relevant Agda code now splits into theorem-facing metatheory modules and the barrier-and-recurrence modules that they explain.

- The theorem-facing modules are:
  - `Metatheory/Extensional.agda`,
  - `Metatheory/KanSubsumption.agda`,
  - `Metatheory/AdjunctionBarrier.agda`.
- On the recurrence side, the key modules are:
  - `Core/Nat.agda`,
  - `Core/AffineRecurrence.agda`,
  - `ObligationGraph/Recurrence.agda`,
  - `Test/Fibonacci.agda`,
  - `Test/MetatheorySmoke.agda`.
- On the sealing side, the key modules are `Saturation/Decomposition.agda`, `Saturation/AbstractionBarrier.agda`, and `Saturation/AbstractionBarrier9.agda`.

The metatheory modules implement the semantic dictionary used throughout the paper. In the extensional/UIP case an obligation contributes no new primitive cost because its semantic space collapses to a contractible one. In the cubical case the relevant notion is weaker and computational: an obligation contributes no new primitive cost when a uniform `hcomp`-style derivation synthesizes a witness from already exported boundary data.

`Metatheory/Extensional.agda` defines binary semantic coherence obligations as path-equality types $p \equiv q$ and proves `UIP-forces-depth-1` together with the paper-facing corollary `history-truncates-to-one`. This is the exact Agda realization of the extensional statement that UIP collapses binary coherence and forces coherence depth 1.

`Metatheory/KanSubsumption.agda` packages the open-box inputs consumed by cubical composition; instantiated at square types, this is exactly the missing-face problem for an arity-3 obligation. In particular it proves:

- `arity3-obligation-syntactically-derivable`,
- `history-beyond-two-algorithmically-subsumed`,
- `arity3-open-box-hfilled`.

This is the mechanized form of the telescopic subsumption claim: once the visible unary and binary faces are fixed, cubical composition uniformly computes a witness for the missing arity-3 face. The mechanization therefore establishes inhabitation by a generic derivation scheme, not singletonhood of the full geometric filler space.

`Metatheory/AdjunctionBarrier.agda` supplies the lower bound. It constructs a local two-point type, its swap autoisomorphism, the induced nontrivial path `swap-path`, and the

contradiction theorem `depth1-insufficient`. Thus the cubical setting does not admit a global depth-one collapse of binary coherence spaces.

5.2. **Barrier discipline and affine recurrence.** Its organizing invariant is that resolved layers are exported through opaque interfaces. At the level of proof architecture, this is the computational form of 2.14: once an obligation has been discharged, later constructions depend only on the exported schema and not on the hidden derivation tree that discharged it. In the barrier modules, this is checked locally for the transitions around steps 8 and 9 by showing that inherited obligations are discharged from the owning layer's opaque record interface alone, without reopening deeper internal derivations.

This realizes the abstraction barrier promised by 2.4. It is the reason historical depth is measured in terms of sealed layers rather than in terms of raw derivation ancestry.

On the counting side, the Agda development makes the modeling discipline explicit. The active primitive interface is represented as the disjoint union of the two most recent schema sets, and the saturation step is stated separately rather than hidden in the syntax of imports. The concrete decompositions in `Saturation/Decomposition.agda` then record, step by step, how the next obligation set splits into contributions from the two most recent sealed layers.

`Core/AffineRecurrence.agda` strengthens the old normalized counting story to the payload-aware law

$$\Delta_{n+1} = \Delta_n + \Delta_{n-1} + 2c,$$

defines the shifted sequence $U_n = \Delta_n + 2c$, and proves that U satisfies the homogeneous Fibonacci recurrence. The zero-payload specialization is linked back to the original `Core/Nat.agda` development, while `ObligationGraph/Recurrence.agda` remains as the stable payload-free corollary surface and `Test/Fibonacci.agda` continues to check the concrete Fibonacci identities.

A useful feature of the mechanization is that it separates three claims that are easy to conflate in prose. The *abstraction-barrier claim* says that historical memory is mediated only by opaque exports. The *metatheoretic depth claim* says that cubical filling uniformly derives witnesses for arity-3 obligations while binary obligations remain genuinely independent in general. The *recurrence claim* says that, once the depth-two window and explicit payload accounting are in place, the cardinality of the next primitive interface is governed by the affine depth-two law above. The current Agda development now addresses all three points.

Remark 5.1 (Scope of the mechanization)**.** The present Agda artifact formalizes both the foundational causes and the combinatorial consequences of the coherence window. It proves that UIP yields depth 1, that cubical arity-3 obligations are algorithmically subsumed by uniform composition, that binary coherence does not collapse globally in the cubical setting, and that explicit payload accounting yields the affine recurrence whose constant shift is Fibonacci. The mathematical arguments of section 3 remain important as explanation and as links to the categorical and topological examples, but they no longer stand in for an absent mechanization of the paper's main depth claims.

6. Conclusion

The paper isolates *coherence depth* as a metatheoretic invariant of framework extension in type theory. In extensional settings with UIP, higher coherences collapse and the invariant is 1. In the fully coupled cubical univalent setting studied here, the invariant is 2. The difference between those two cases is not cosmetic: under uniform payload size it changes the recurrence law of integration from one-step affine growth to shifted Fibonacci scaling.

The mechanism behind the recurrence is structural. For globally acting foundational extensions, maximal interface density, enforced as a canonicity requirement, identifies sealing cost with the size of the active interface. The Integration Trace Principle decomposes each new public interface into the layer's own payload together with the discharged obligations that make it cohere with prior exports. In the cubical univalent case, telescopic subsumption shows that all older comparisons are algorithmically synthesized from the adjacent exported traces, so the primitive active interface is generated by the previous two layers; under constant payload size this yields an affine depth-two recurrence whose shifted form is Fibonacci.

From this viewpoint the appearance of the Golden Ratio is not an empirical regularity of one implementation and not a matter of aesthetic numerology. It is the characteristic root of the homogeneous recurrence recovered after separating the constant payload contribution from the depth-two coherence memory.

The current Cubical Agda artifact now matches that story. It proves the extensional collapse, the cubical upper bound, the lower bound against depth 1, and the affine recurrence that shifts to Fibonacci. The next natural step is therefore not to formalize the depth theorem package itself, but to connect this metatheory layer more tightly to the repository's oracle and candidate-generation components.

References

[CCHM18] Cyril Cohen, Thierry Coquand, Simon Huber, and Anders Mörtberg. Cubical type theory: a constructive interpretation of the univalence axiom. In *21st International Conference on Types for Proofs and Programs (TYPES 2015)*, 2018.

[CHM18] Thierry Coquand, Simon Huber, and Anders Mörtberg. On higher inductive types in cubical type theory. In *Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science*, 2018.

[Hus94] Dale Husemoller. *Fiber Bundles*, volume 20 of *Graduate Texts in Mathematics*. Springer, 3 edition, 1994.

[KvR19] Nicolai Kraus and Jakob von Raumer. Coherence via well-foundedness. In *Proceedings of the 34th Annual ACM/IEEE Symposium on Logic in Computer Science*, 2019.

[Lan63] Saunders Mac Lane. Natural associativity and commutativity. *Rice University Studies*, 49(4):28–46, 1963.

[Lum10] Peter LeFanu Lumsdaine. Weak ω-categories from intensional type theory. In *Typed Lambda Calculi and Applications*, volume 6690 of *Lecture Notes in Computer Science*, pages 172–187, 2010.

[Sta63] James D. Stasheff. Homotopy associativity of H-spaces I, II. *Transactions of the American Mathematical Society*, 108(2):275–312, 1963.

[Uni13] Univalent Foundations Program. *Homotopy Type Theory: Univalent Foundations of Mathematics*. Institute for Advanced Study, 2013. URL: `https://homotopytypetheory.org/book`.

[vdBG11] Benno van den Berg and Richard Garner. Types are weak ω-groupoids. *Proceedings of the London Mathematical Society*, 102(2):370–394, 2011.

[VMA19] Andrea Vezzosi, Anders Mörtberg, and Andreas Abel. Cubical agda: a dependently typed programming language with univalence and higher inductive types. In *Proceedings of the 24th ACM SIGPLAN International Conference on Functional Programming*, 2019.